

Software Design

ACRONYMS

API	Application Programming Interface
AOD	Aspect-Oriented Design
CBD	Component-Based Design
CRC	Class Responsibility Collaborator (Or Collaboration)
DFD	Data Flow Diagram
DSL	Domain-Specific Language
ERD	Entity Relationship Diagram
FOSS	Free And Open Source Software
IDL	Interface Description Language
MBD	Model-Based Design
MDD	Model-Driven Design
OO	Object-Oriented
PDL	Program Design Language
SDD	Software Design Description
SoC	Separation of Concerns
UML	Unified Modeling Language

INTRODUCTION

This chapter considers software design from several perspectives — focusing on basic concepts, context and processes, software design qualities and strategies, and recording and evaluating designs.

Design is used in distinct but closely related ways to refer to (1) the *discipline* (“use of scientific principles, technical information, and imagination in the definition of a software system to perform [prespecified] functions with maximum economy and efficiency”) [11]; (2) the *processes* for performing within that discipline; (3) the *result* of applying that

discipline; and (4) the *stage* in the life cycle of a software system during which those processes yield those results.

A *software design description* (SDD) documents the result of software design. It is a “representation of software created to facilitate analysis, planning, implementation, and decision-making. The software design description is used as a medium for communicating software design information and can be thought of as a blueprint or model of the system” [11].

The SDD, which may take many forms, encompasses the refinement of that software into components, the organization of those components, and the definition of interfaces among them and between the software and the outside world — to a level of detail that enables their construction.

Software design, viewed as a life cycle activity, is the application of software engineering discipline in which software requirements are analyzed to define the software’s external characteristics and internal structure as the basis for the software’s construction.

Software design takes place in three stages:

- architectural design of the software system
- high-level or external-facing design of the system and its components
- detailed or internal-facing design

Architectural design is a part of architecting, discussed in the Software Architecture KA.

BREAKDOWN OF TOPICS FOR SOFTWARE DESIGN

The breakdown of topics for the Software Design KA is shown in Fig. 3.1.

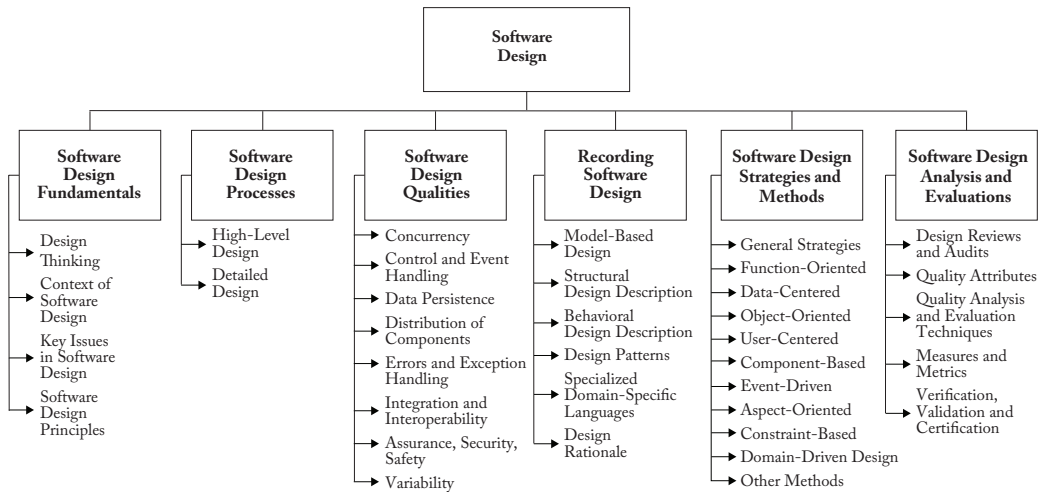


Figure 3.1. Breakdown of topics for the Software Design KA

1. Software Design Fundamentals [3*][4*]

The concepts, notions and terminology introduced here form a basis for understanding the role and scope of software design.

1.1. Design Thinking [3*, c1-3] [4*, c1-2]

Design is all around us, in the things and organizations that have been created to meet a need or solve a problem.

In a general sense, design can be viewed as a form of problem-solving. For example, the concept of a *wicked problem* — a problem with no definitive solution — is interesting in terms of understanding the limits of design. Many other notions and concepts help us understand design in its general sense: goals, constraints, alternatives, representations and solutions. (See also *Design as a Problem-Solving Activity* in Engineering Foundations KA.)

Design thinking comprises two essentials: (1) understanding the need or problem and (2) devising a solution. Ross, Goodenough and Irvine offer an elaboration of design thinking appropriate to software:

This process consists of five basic steps: (1) crystallize a purpose or objective; (2) formulate a concept for how the purpose can be achieved;

(3) devise a mechanism that implements the conceptual structure; (4) introduce a notation for expressing the capabilities of the mechanism and invoking its use; (5) describe the usage of the notation in a specific problem context to invoke the mechanism so the purpose is achieved. [20]

This is particularly appropriate because much of software design consists of creating the necessary *vocabulary* to express a problem, express its solution and implement that solution. The steps emphasize the linguistic nature of software design problem solving. This is a recurring pattern we see throughout high-level design, detailed design and architecting (see *Architecting in the Large* in Software Architecture KA). Therefore, Software Design is a practical process of transforming a problem statement into a solution statement. Software design shares commonalities with other kinds of design. Design can be further understood via *design theory* [8].

1.2. Context of Software Design [4*, c13-14] [21*, c19-20]

Software design is an important part of the software development process. To understand the role of software design is to see how it fits into the software development life cycle (see

Software Process KA). To understand that context, it is important to understand the major characteristics and roles of software requirements, software construction, software testing, and software maintenance. The context varies with many factors, including degree of formality and stage of the life cycle.

Software design is the transformation of customer and other requirements, needs, and concerns into implementable design specifications. Its contexts include the following:

- Software Design's relationship with software requirements: The requirements establish a set of problems that the software design must solve.
- Software Design's relationship with software architecture: In cases where an architecture has been established, that architecture constrains the design by capturing fundamental aspects of the system: such as its major components and their interconnections, application programming interfaces (APIs), styles and patterns to be used, and architectural principles to be observed and enforced.
- Software Design's relationship with software construction: The software design must provide a guide to implementors on building the system.
- Software Design's relationship with software testing: Software design provides a foundation for an overall testing strategy and test cases that ensure that the design is properly implemented and operates as intended.

1.3. Key Issues in Software Design [3*, p69ff] [4*, c17.1.1] [21*, c6-7]

Many key issues must be dealt with when designing software. Some are quality *concerns* that all software must address (performance, security, reliability, usability, maintainability, etc.). Another important issue is how to refine, organize, interconnect and package software components. These issues are so fundamental that all design approaches address them in one

way or another. (See topic *Stakeholders and Concerns* in Software Architecture KA, section 1.4 *Software Design Principles*, and topic 5 *Software Design Strategies and Methods*.)

In contrast, other issues “deal with some aspect of software’s behavior that is not in the application domain, but which addresses some of the supporting domains” [2]. Such issues, which often crosscut the system’s functionality, are referred to as *aspects*, which “tend not to be units of software’s functional decomposition, but rather to be properties that affect the performance or semantics of the components in systemic ways” [12].

1.4. Software Design Principles [4*, c4.2]

A *principle* is “a fundamental truth or proposition that serves as the foundation for a system of belief or behavior or for a chain of reasoning.” [Oxford English Dictionary]

Design principles provide direction or guidance for making decisions during design. Some principles originated during the early days of software engineering, others even predate the discipline, deriving from best practices in engineering unrelated to software. (See Engineering Foundations KA.) Decision making can also be assisted by quantitative methods, such as discussed in Software Engineering Economics KA. Software design principles are key notions that provide the basis for many different software design concepts, approaches and methods. The principles listed below apply to any of the three stages of design. Many of these principles are interrelated. Whether alone or used in combination with other principles, they are reflected elsewhere in software design to produce many concepts and constructs found in design capture, strategies and methods. This is itself an application of the design thinking process above. Software design principles include the following:

- *Abstraction* is “a view of an object that focuses on the information relevant to a particular purpose and ignores the remainder of the information” [11].“The

abstraction principle . . . helps to identify essential properties common to superficially different entities” [20]. (See also topic *Abstraction* in the Computing Foundations KA.)

- *Separation of concerns (SoC)*. A *design concern* is an “area of interest with respect to a software design” [11] that is relevant to one or more of its stakeholders. By identifying and separating concerns, the designer can focus on each concern for the system in isolation about which Dijkstra said “even if not perfectly possible, [SoC] is yet the only available technique for effective ordering of one’s thoughts” [5]. (See also topic *Stakeholders and Concerns* in Software Architecture KA.)
- *Modularization* (or *refinement* or *decomposition*) structures large software as comprising smaller components or units. Each component is named and has well-defined interfaces for its interactions with other components. Smaller components are easier to understand and, therefore, to maintain. There are numerous modularization strategies. (See topic 5 Software Design Strategies and Methods.) Traditionally, the goal is to place distinct functionalities and responsibilities in different components. David Parnas advocated that each module in a system should have a single responsibility [17]. One way to think of modularization is as a special case of more general strategies, such as separation of concerns or divide and conquer. (see topic Problem-Solving Techniques in Computing Foundations).
- *Encapsulation* (or *information hiding*) builds upon the principles of abstraction and modularization so that nonessential information is less accessible, allowing users of the module to focus on the essential elements at the interface.
- *Separation of interface and implementation* is an application of encapsulation that involves defining a component by specifying its public interfaces, which are known to and accessible to clients; isolating the use of a component from

the details of how that component is built. (See *Encapsulation* (or *information hiding*) above.)

- *Coupling* is defined as “a measure of the interdependence among modules in a computer program” [11]. Most design methods advocate that modules should be loosely or weakly coupled.
- *Cohesion* (or *localization*) is defined as “a measure of the strength of association of the elements within a module” [11]. Cohesion highlights organizing a module’s constituents based on their relatedness. Most design methods advocate that modules should maximize their cohesion/locality.
- *Uniformity* is a principle of consistency across software components — common solutions should be produced to address common or recurring problems. These include naming schemes, notations and syntax, interfaces that define access to services and mechanisms, and ordering of elements and parameters. This can be achieved through conventions such as rules, formats and styles.
- *Completeness* (or *sufficiency*) means ensuring that a software component captures the important characteristics of an abstraction and leaves nothing out. Completeness takes various forms, perhaps the most important of which is design completeness against requirements: a design should be sufficient for designers to demonstrate how requirements will be met and how subsequent work will satisfy those requirements. Design should be complete with respect to the modes and states of the software.
- *Verifiability* means that information needed to verify the design against its requirements and other constraints is available. This is relevant for any software but is of particular importance for high-assurance software, such as software where security, reliability or safety-critical concerns are present. An SDD should be sufficient as a basis for verifying a design. (See Software Testing KA and Software Quality KA.)

- *Other design principles.* With the increased appearance of autonomous systems, the use of machine learning and artificial intelligence, and, generally, systems with widening social impacts, approaches to *Ethically Aligned Design* have been developed to address concerns including universal human values, political self-determination, and data agency and technical dependability [9]. The general principles of Ethically Aligned Design are human rights, well-being, data agency, effectiveness, transparency, accountability, awareness of misuse, and competence.

2. Software Design Processes

[4*, c3] [21* c2, c7]

Software design is generally considered a multistage process or activity. Software design can be divided into the following stages or phases. When necessary, we distinguish the phase from the general activity:

- Architectural design stage
- High-level design stage
- Detailed design stage

The architectural design stage addresses the fundamentals of the system as a whole and in relation to its environment (see Software Architecture KA).

The high-level design stage is *outward-facing* — developing the top-level structure and organization of the software, identifying its various components and how that software system and its components interact with the environment and its elements.

The detailed design stage is *inward-facing* — specifying each component in sufficient detail to facilitate its construction and to meet its outside obligations, including how software components are further refined into modules and units.

Each stage reflects the basic pattern outlined in section 1.1 *Design Thinking*.

Not all stages are found in every software process. However, when present, each stage creates an obligation upon the next

stage regarding the software which is under development.

Although software developers generally follow similar guidelines for what happens in each stage, there are no strict boundaries between stages regarding what must be done and when. For example, for many software systems, the choice of an algorithm to sort data will be deferred to programmers, within the constraints and guidance provided by the system's requirements, its architecture description or design specifications. However, for another software system, the existence of a suitable algorithm could be architecturally significant and must be determined early in the life cycle. Without that algorithm, there is no possibility of constructing the software to meet its requirements.

Some rules of thumb for each stage include the following:

- The architectural design stage defines a computational model, the major computational elements, and the important protocols and relationships among them. This stage develops strategies to address crosscutting concerns, such as performance, reliability, security and safety, and articulation of crosscutting decisions, including system-wide styles (e.g., a transactional n-tier style versus a pipes and filters style, together with the rationale for such decisions).
- The high-level design stage includes identification of the primary computational elements and significant relationships among them, with a focus on each major component's existence, role and interfaces. That definition should be sufficiently detailed to allow designers or programmers of client components to correctly and efficiently access each service's capabilities — without having to read its code.
- The detailed design stage defines each module's internal structure, focusing on detailing and justifying choices of algorithms, data access and data representation. The detailed design specifications should be sufficient to allow programmers

to code each module during construction (see Software Construction KA). The code is a representation of the solution that is sufficiently detailed and complete that a compiler (or interpreter) can execute it.

2.1. High-Level Design [3*, c5] [4*, c6]

High-level design specifies the interaction of a system's major components with one another and with the environment, including users, devices and other systems. High-level design addresses the following:

- External events and messages to which the system must respond
- Events and messages which the system must produce
- Specification of the data formats and protocols for events and messages
- Specification of the ordering and timing relationships between input events and messages, and output events and messages
- Tracing and analysis of end-to-end transactions and event threads
- Data persistence (how data is stored and managed)

High-level design is undertaken within the envelope established by the system's software architecture (if any). Each of the above may be guided or constrained by architecture directives. For example, event signaling and messaging will use the protocols and modes of interaction established by the architecture. Data formats and protocols will use data and communication standards specified by the architecture. Absent an explicit architecture design stage, some of these directives will be established by the software requirements or decided during high-level design.

2.2. Detailed Design [4*, c14]

The detailed design stage proceeds within the constraints established by the high-level design. It specifies major system components' internal characteristics, internal modules and their interconnections to other modules, services

and processes they provide, computing properties, algorithms, and data access rules and data structures. This includes the following:

- Refinement of major system components into modules or program units, including opportunities for using off-the-shelf components and application frameworks
- Allocation of design responsibilities to modules and program units
- Interactions among modules
- Scope and visibility among components, modules and program units
- Component modes, component states and transitions among them
- Data and control interdependencies
- Data organization, packaging and implementation
- User interfaces
- Requisite algorithms and data structures

3. Software Design Qualities [4*, c4]

Software requirements and architecture directives are intended to guide software toward certain characteristics or *design qualities*. Design qualities are an important subclass of concerns (see topic *Stakeholders and Concerns* in Software Architecture KA). One role of design principles (see section 1.4 *Software Design Principles*) is to help software achieve these qualities. Among the characteristics of interest to designers are the following:

3.1. Concurrency [21*, c17]

Design for concurrency concerns how software is refined into concurrent units such as processes, tasks, and threads and the consequences of those decisions with respect to efficiency, atomicity, synchronization and scheduling.

3.2. Control and Event Handling [21*, c21]

Event handling is concerned with how to organize control flow as well as how to handle reactive and temporal events through various mechanisms including synchronization, implicit invocation and callbacks.

3.3. *Data Persistence* [21*, c6, c16]

Data persistence concerns the storage and management of data throughout the system.

3.4. *Distribution of Components* [21*, c17]

Distribution concerns how software components are distributed across hardware (including computers, networks and other devices) and how those components communicate while meeting performance, reliability, scalability, availability, monitorability, business continuity and other expectations.

3.5. *Errors and Exception Handling, Fault Tolerance* [21*, c11]

This concern pertains to how to prevent, avoid, mitigate, tolerate and process errors and exceptional conditions.

3.6. *Integration and Interoperability* [4*, c11, c14, c16]

This issue arises at the enterprise or system-of-systems level or for any complex software when heterogeneous systems or applications need to interwork through exchanges of data or accessing one another's services. Within a software system, the issue arises when components are designed using different frameworks, libraries or protocols.

3.7. *Assurance, Security, and Safety* [21*, c10-14]

High assurance spans a number of software qualities, including security and safety concerns, pertaining to whether the software behaves as intended in critical situations, such as in the face of hazards. Security becomes a key concern for distributed applications where components communicate using different protocols and media. *Design for security* concerns how to prevent unauthorized disclosure, creation, change, deletion, or denial of access to information and other resources in the face of attacks upon the system or violations of system

policies to limit damage; provide continuity of service; and assist repair and recovery. *Design for safety* pertains to managing the software's behavior in circumstances which might lead to harm to or loss of human life or damage to property or the environment.

3.8. *Variability* [7*]

Variability concerns permissible variations in a software system. It is a fundamental aspect of most software [6]. It is the ability to create software system variants for different market segments or contexts of use.

Interest in variability first arose in software product lines and system families, to accommodate and manage deployment of multiple variants such as for different organizations or markets. (See appendix B 6, Standards for product line, methods and tools). It is also relevant to software ecosystems and context-aware software. (See also 3.5 Reuse in Construction, Software Construction KA.)

Feature models are used to gather requirements and dependencies into bundles. (See Feature-Driven Development, under topic 4.1 Agile Methods in Software Engineering Models and Methods KA).

4. *Recording Software Designs* [1*] [4* c7-8]

The outcome of design processes is accumulated knowledge and work products recording that knowledge. Work products of software design capture (1) aspects of the problems to be solved, using the vocabulary of the domain; (2) a solution vocabulary for solving the design problems (see section 1.1 *Design Thinking*); (3) the major decisions that have been taken; and (4) explanations of the rationale for each nontrivial decision. Recording the rationale for important decisions enhances the software product's long-term maintainability when modifications or enhancements are considered (see section 4.6 *Design Rationale*). These work products, often termed *design descriptions* or *design specifications*, can take the form of texts, diagrams, models and prototypes

that comprise the blueprints of the software to be implemented.

A fundamental aspect of software design is communication about the design among designers, and to customers, implementers and other stakeholders. This is the case whether the software is developed using agile, traditional or formal methods. The communication will vary depending upon the target audience, the level of detail being communicated, and relevance to the concerns of the stakeholders. For example, when using traditional or formal methods, the design often evolves through a progression of design descriptions, while in agile approaches the evolving design may be implicit in the minds of developers and only explicit as code. While the latter approach supports the agility of developers, other stakeholders, such as those concerned with requirements, certification, testing and quality assurance may need explicit design information to do their work. Therefore, projects should make conscious decisions about which design specifications are needed based upon stakeholder audience, subject and intended usage.

Designers can analyze and evaluate these work products to determine whether the design can meet the requirements and constraints on the software. Software design also examines and evaluates alternative solutions and trade-offs. In addition to using them as inputs and as the starting point for construction and testing, stakeholders can use the design work products to plan subsequent activities, such as system verification and validation.

As design concepts evolve, so do their representations (see section 1.1 *Design Thinking*); part of the design process involves creating appropriate vocabularies for problems and solutions. An informal sketch may be most appropriate for the early stages. It is useful to distinguish in-process (“working”) specifications from final design products. The former are produced *by* the design team *for* the design team; the latter may be produced for known stakeholders or even for an unknown future audience.

Many notations exist to represent software design artifacts. Software design is often carried out using multiple types of notation. Two broad areas of concern are software structures and software behaviors. Some are used to describe a design’s structural organization, others to represent the software’s intended behavior. Below, they are categorized as notations for structural and behavioral concerns (see section 4.2 *Structural Design Descriptions* and section 4.3 *Behavioral Design Descriptions*, respectively). Certain notations are used mostly during architectural design and others mainly during detailed design; some are useful throughout all stages of software design. Some notations are closely tied to the context of specific design methods (see Software Design Strategies and Methods KA).

The Unified Modeling Language (UML) is a widely used family of notations addressing both structural and behavioral concerns and is used in all design stages, from architectural through detailed design [1].

4.1. *Model-Based Design* [4*, c7.3] [21*, c5.5]

Over the history of software engineering, including architecture and design, there has been an evolution from document-based artifacts to model-based artifacts. *Model-Based Design* (MBD) is an approach to recording designs where models play an important role.

This trend reflects the limitations of document-based artifacts and the increased capabilities of automated tools. Document-based artifacts use natural language and informal diagrams to convey designers’ intentions, which might introduce ambiguity and incompleteness. Even when documents use well-defined formats, relevant information might be spread across documents, making understandability and analysis difficult. With MBD, appropriate tooling can gather and organize relevant information for use by designers and other stakeholders in an accessible form.

Modern tools have accelerated the trend from document to model-based artifacts.

Tooling enables animation or simulation of various software aspects, analyses of what-if scenarios and trade-offs, and rapid prototyping. Tooling also facilitates continuous testing and integration approaches, enhanced and interactive traceability, and knowledge capture and management, which are inefficient or even infeasible with document-based approaches.

Model-driven development (MDD) is a development paradigm that uses models as the development process' *primary* artifacts (see *Software Engineering Models and Methods* KA).

4.2. *Structural Design Descriptions*

[1*, c4-14][4*, c7, c10] [7*, c4] [21*, c5.3]

The following types of notation, most of which are graphical, are used to represent the structural aspects of a software design — that is, they are used to describe the major components and how they are interconnected (static view) and the allocation of responsibilities to components and modules:

- Class and object diagrams are used to represent a set of classes and objects and their interrelationships.
- Component diagrams are used to represent a set of *components* (replaceable elements of a system that conform to and provide the realization of a set of interfaces) and their interconnections. Component models evolved from earlier module interconnection languages into the package systems of programming languages like Ada and Java and the sophisticated module systems of current functional language systems such as Haskell and Coq.
- Class responsibility collaborator cards (CRCs) are used to denote the names of components (classes), their responsibilities and the components they interact with to meet those responsibilities.
- Deployment diagrams are used to represent a set of physical nodes and their interconnections to model the physical aspects of software as deployed on hardware.

- Entity relationship diagrams (ERDs) are used to represent conceptual, logical and physical models of data as stored in information repositories or as a part of interface descriptions.
- Interface description languages (IDLs) are programming-like languages used to define the interfaces (names and types of exported operations) of software components.
- Structure charts are used to describe the calling structure of programs (that is, they show which modules call, and are called by, which other modules).

4.3. *Behavioral Design Descriptions*

[1*, c15-24] [4*, c9-10] [7*, c5] [21*, c5.4]

The following notations and languages, some graphical and some textual, are used to describe the dynamic behavior of software systems and their components. Many of these notations are useful mostly, but not exclusively, during detailed design. Moreover, behavioral descriptions can include rationale for design decisions (see section 4.6 *Design Rationale*).

- Activity diagrams are used to show flow of a computation from activity to activity. They also can represent concurrent activities, their inputs and outputs and opportunities for concurrency.
- Interaction diagrams characterize the interaction among a group of objects. There are two major kinds of interaction diagrams: communication (or collaboration) diagrams and sequence diagrams. Communication diagrams show interactions among objects with an emphasis on their links and the messages they exchange on those links. Sequence diagrams show interactions among objects, with an emphasis on the temporal ordering of messages passed among those objects.
- Data flow diagrams (DFDs) are used to show data flow among computing elements. A DFD provides “a description based on modeling the flow of information around a network of operational

elements, with each element making use of or modifying the information flowing into that element” [4]. DFDs have other uses, such as security analysis, as they identify possible paths for attack and disclosure of confidential information.

- Decision tables and diagrams are used to represent complex combinations of conditions and actions.
- Flowcharts are used to represent the flow of control and the sequence of associated actions.
- State (transition) diagrams and statecharts are used to show transitions from state to state and how a component’s behavior changes based on its current state and response to input events.
- Formal specification languages are predominantly textual languages founded upon basic notions from mathematics (for example, type, set, sequence, logical proposition) to rigorously and abstractly define software component interfaces and behavior, often in terms of pre- and post-conditions, invariants, type checking, and computational models (see section *Formal Methods* in *Software Engineering Models and Methods KA*).
- Pseudocode and program design languages (PDLs) are structured, programming language-like notations used to describe a procedure’s processing behavior, generally at the detailed design stage. The use of these languages is less common today but is still found in the documentation of algorithms.

4.4. *Design Patterns and Styles*

[3*, c12] [4*, c15] [7*, c1-2] [21*, c7.2]

Succinctly described, a pattern is “a common solution to a common problem in a given context” [7]. Design patterns include the following:

- Creational patterns (e.g., builder, factory, prototype, singleton)
- Structural patterns (e.g., adapter, bridge, composite, decorator, façade, flyweight, proxy)
- Behavioral patterns (e.g., command,

interpreter, iterator, mediator, memento, observer, peer-to-peer, publish-subscribe, state, strategy, template, visitor)

Design patterns can be used to reflect idioms that have proven useful in solving particular design problems in the past, establish a solution vocabulary, and document and explain design decisions. They arise at all stages of design, including architectural design. Often architectural styles can be viewed as patterns “in the large,” describing common solutions to architecture-level problems that pervade the software. (See also topic 2.2 *Architecture Styles and Patterns*, *Software Architecture KA*).

4.5. *Specialized and Domain-Specific Languages* [21*, c15]

Not every design representation falls easily into the structure/behavior dichotomy. For example, user interface design mixes the structural layout of what a user might see with the behavioral logic of sequencing screens based upon user actions. Specialized concerns such as safety and reliability often have their own forms of representation that have evolved among specialists in those communities [21].

A recent trend has been the maturing of *domain-specific languages* (DSLs) and widely available tools to develop them. In this approach, part of the design process is codifying concepts and constructs of a specific application domain to create a computer language for that domain so that representing the design using these constructs leads to an animated or executable implementation. DSLs blur the lines among modeling languages, design languages and programming languages in this approach.

There are DSLs and supporting tools for domains such as simulation; real-time, reactive and distributed systems; game development; user interfaces; test development; and language processing tools. The growth of DSLs has been facilitated by increasingly powerful grammar-driven tools that, given a language definition, can generate a graphical user interface, syntax checkers, code generators, compilers and linkers for the specialized language.

4.6. *Design Rationale* [3*, c16][4*, c12] [21*, c6.1]

A useful design outcome is insight into and explicit documentation of the major decisions taken, along with an explanation of the rationale for each decision. Design rationale captures *why* a design decision was made. This includes prior assumptions made, alternatives considered, and trade-offs and criteria analyzed to select one approach and reject others.

Although the reasons for decisions are likely to be obvious to the current design team, they can be less obvious to those who modify or maintain the system after deployment. Recording the rationale enhances the software product's long-term maintainability. Continuing to capture the rationale for changes during maintenance also contributes to the software's viability.

It can also be useful to capture rejected decisions and the reasons for rejection. Capturing these rationales can enable a team to revisit a previously rejected decision when assumptions, requirements or constraints change. The importance of rationale is visible, for example, in free and open-source software (FOSS) projects, which often involve large, distributed teams of developers with frequent turnover.

Design rationale may be captured as part of a software design description or as a companion artifact. Often rationale is captured in text, but other forms of representation can also be used, such as graphs that portray a design as an interconnected network of decisions.

5. **Software Design Strategies and Methods** [21*, c3]

Various strategies and methods exist to structure and guide the design process; many of these evolved from programming styles or paradigms. In addition to embodying one or more general strategies, most design methods focus on making one or more design concepts (whether objects, methods or events) prominent as organizing themes for the software. These themes then guide the designers as to what to focus on first, how to proceed, and how to structure modules.

5.1. *General Strategies* [4*, c13]

Some often-cited examples of general strategies useful in the design process include divide-and-conquer and stepwise refinement strategies; top-down vs. bottom-up strategies; strategies using heuristics, patterns and pattern languages; and iterative and incremental approaches.

5.2. *Function-Oriented (or Structured) Design* [4*, c9]

This is one of the classical software design methods. It focuses on refinement (or decomposition) to identify major software functions, elaborating them in a top-down manner. Structured design often follows structured analysis, producing DFDs and associated process descriptions. Various tools enable the automated translation of DFDs into high-level designs.

5.3. *Data-Centered Design* [4*, c9] [21*, c5.4.1]

Data-centered design starts from the data structures a program manipulates rather than from the functions it performs. The software designer specifies the input and output data structures and then develops program units that transform inputs into outputs. Various heuristics have been proposed to deal with special cases, such as cases where there is a mismatch between the input and output structures.

5.4. *Object-Oriented Design* [4*, c10]

Numerous software design methods based on objects have been proposed. The field has evolved from the early object-oriented design of the mid-1980s (where nouns depict objects; verbs depict methods; and adjectives depict attributes), where inheritance and polymorphism play key roles, to the field of component-based design (CBD), where meta-information can be defined and accessed (through reflection, for example). Although OOD's roots stem from the concept of data abstraction, responsibility-driven design has been proposed as an alternative underlying

principle of OOD. Often design strategies are provided with mnemonics such as SOLID (Single-responsibility, Open-closed, Liskov substitution, Interface segregation, and Dependency inversion) principles of class design and SOFA (Short, One thing, Few arguments and Abstraction level consistency) principles for method design.

5.5. *User-Centered Design* [3*, c9]

User-centered design is more than a design method; it is a multidisciplinary approach emphasizing a deep understanding of users and their needs as the basis for designing user experiences within the context of their organization and the tasks to be accomplished. It involves gathering user requirements, creating a user flow of tasks and decisions, creating prototypes or mockups representative of user interfaces, and evaluating the design solution against original requirements [16].

5.6. *Component-Based Design (CBD)* [1*, c25, c29] [4*, c11, c16] [21*, c16]

CBD decomposes a software system into one or more standalone components that communicate only on well-defined interfaces and conform to a system-wide standard component model. A software component is an independent unit, having well-defined interfaces and dependencies that can be composed and deployed independently. CBD addresses issues related to providing, developing and integrating such components to improve reuse. CBD often emphasizes common APIs for all components and specialized APIs for specific services or responsibilities.

5.7. *Event-Driven Design* [21*, c5.4.2]

Event-driven design is an approach where a system or component invokes its operations in reaction to events (indirect invocation) [15]. Publish/subscribe messaging (broadcasting) is often used as means of transporting events via the network to all interested subscribers. Publish/subscribe keeps the producers and

consumers decoupled using a message broker with channels called topics. This differs from Point-to-point messaging where senders and receivers need to know each other to deliver and receive a message. Different types of event processing exist, i.e. simple event processing, event stream processing and complex event processing. Message-based systems frequently incorporate identifiable senders and receivers within the design. Event-driven systems may not identify senders and receivers explicitly — instead each module produces events while listening for any events they care about or need to respond to [14]. “Anonymous” asynchronous message and event processing are good strategies for scalable systems.

5.8. *Aspect-Oriented Design (AOD)* [1*, c10] [21*, c31]

AOD is a method by which software is constructed using aspects to implement the cross-cutting concerns and extensions identified in software requirements [12]. AOD evolved from object-oriented design and programming practices. Although it has yet to become a widespread design or programming paradigm, the aspect-oriented perspective is frequently used in application frameworks and software libraries where parameters of the framework or library can be configured with aspect declarations.

5.9. *Constraint-Based Design* [3*, c11]

Constraints’ role in the design process is to limit the size of a design space to exclude infeasible or unacceptable alternatives. Constraints accelerate design because they force a few early decisions. The constraints can reflect limits imposed on the hardware, software, data, operational procedures, interfaces or anything that affects the software. The constrained design space can then be explored with search or backtracking methods. Constraint-based design approaches are used in user interface design, gaming and other applications. In general, constraint satisfaction problems can be computationally intractable; however, various kinds

of constraint-based programming can be used to approximate or solve constraint problems.

5.10. *Domain-Driven Design* [4*, c13.6.2, c18.3]

Domain-driven design is a method in which the designer uses a domain-specific language shared with analysts and other stakeholders to describe the target software system. Through this shared language, objects, roles, events, and activities specified in the software requirements can be expressed in the software design descriptions. (See the Requirements KA).

5.11. *Other Methods* [21*, c18–c21]

Other approaches to design exist (see Software Engineering Models and Methods KA). For example, iterative and adaptive methods implement software increments and reduce the emphasis on rigorous software requirements and design.

Service-oriented methods builds distributed software using web services executed on distributed computers. Software systems are often constructed using services from different providers interconnect with standard protocols (e.g., HTTP, HTTPS, SOAP) designed to support service communication and service information exchange.

6. Software Design Quality Analysis and Evaluation [4*, c7] [21*, c24]

6.1. *Design Reviews and Audits* [4*, c5.3]

Design reviews are intended as comprehensive examinations of a design to assess concerns such as status or degree of completion, coverage of requirements, open or unresolved issues and potential problems. A design review can be undertaken at any stage of design. Design reviews can be conducted by the design team, by an independent third party or other stakeholder. A design audit is more narrowly focused on a set list of characteristics (e.g., a functional audit). (See also section 2.3 *Reviews and Audits* in Software Quality KA).

6.2. *Quality Attributes* [21*, c24]

Various attributes contribute to the quality of a software design, including various “ilities” (modularity, maintainability, portability, testability, usability) and “nesses” (correctness, robustness). Qualities are a major subset of concerns (see topic *Stakeholders and Concerns* in Software Architecture KA). Some qualities can be observed at runtime (e.g., performance, security, availability, functionality, usability); others cannot (e.g., modifiability, portability, reusability, testability); some (e.g., conceptual integrity, correctness, completeness) are observable in the design of the software.

6.3. *Quality Analysis and Evaluation Techniques* [21*, c24]

Various tools and techniques can help in analyzing and evaluating software design quality. (See also topic Software Quality Tools in Software Quality KA.)

- Software design reviews include informal and rigorous techniques to determine software qualities based on SDDs and other design artifacts for example, architecture reviews, design reviews and inspections; scenario-based techniques; requirements tracing.
- Static analysis: formal or semiformal static (nonexecutable) analysis that can be used to evaluate a design (for example, fault-tree analysis or automated cross-checking). Design vulnerability analysis (for example, static analysis for security weaknesses) can be performed if security is a concern. Formal design analysis uses mathematical models that allow designers to predict the behavior and validate the performance of the software instead of having to rely entirely on testing. Formal design analysis can be used to detect residual specification and design errors (perhaps caused by imprecision, ambiguity, and sometimes other kinds of mistakes). (See also Software Engineering Models and Methods KA.)
- Simulation and prototyping: dynamic

techniques to evaluate a design (for example, performance simulation or feasibility prototypes).

6.4. Measures and Metrics

[4*, c5, c17] [21*, c24.5]

Measures can be used to assess or to quantitatively estimate various aspects of a software design; for example, size, structure, or quality. Most measures that have been proposed are based upon the approach used for producing the design (see topic 5 Software Design Strategies and Methods). These measures are classified in two broad categories:

- Function-based (structured) design measures: measures obtained by analyzing functional decomposition; generally represented using a structure chart (or hierarchical diagram) on which various measures can be calculated.
- Object-oriented design measures: the design structure is typically represented as a class diagram, on which various measures can be computed. Measures on

the properties of the internal content of each class can also be calculated. Object-oriented measures also consider the complexity of the code based on the lines of code per method or the number of messages sent.

6.5. Verification, Validation, and Certification

[21*, c7-8]

Systematic analysis or evaluation of the design plays an important role in each of these three areas:

- verification: to confirm that the design satisfies stated requirements;
- validation: to establish that the design will allow the system to meet the expectations of its stakeholders, including customers, users, operators and maintainers;
- certification: third-party attestation of conformity of design to its overall specification and intended usage.

(See also section 2.2 *Verification and Validation* in Software Quality K.A.)

MATRIX OF TOPICS VS. REFERENCE MATERIAL

	Booch et al. [1*]	Brooks [3*]	Budgen [4*]	Gamma et al. [7*]	Sommerville [21*]
1. Software Design Fundamental		c1-3	c1-2		
1.1. Design Thinking		c1-3	c1-2		
1.2. Context of Software Design			c13-14		c19-20
1.3. Key Issues in Software Design		p69ff	c17.1.1		c6-7
1.4. Software Design Principles			c4.2		
2. Software Design Processes			c3		c2,c7
2.1. High-Level Design		c5	c6		
2.2. Detailed Design			c14		
3. Software Design Qualities			c4		
3.1. Concurrency					c17
3.2. Control and Event Handling					c21
3.3. Data Persistence					c6,c16
3.4. Distribution of Components					c17

3.5. <i>Errors and Exception Handling Fault Tolerance</i>					c11
3.6. <i>Integration and Interoperability</i>			c11, c14, c16		
3.7. <i>Assurance, Security and Safety</i>					c10-14
4. Recording Software Designs	c1-3		c7-8		
4.1. <i>Model-Based Design</i>			c7.3		c5.5
4.2. <i>Structural Design Descriptions</i>	c4-14		c7, c10	c4	c5.3
4.3. <i>Behavioral Design Descriptions</i>	c15-24		c9-10	c5	c5.4
4.4. <i>Design Patterns and Styles</i>		c12	c15	c1-2	c7.2
4.5. <i>Specialized and Domain-Specific Languages</i>					c15
4.6. <i>Design Rationale</i>		c16	c12		c6.1
5. Software Design Strategies and Methods					c3
5.1. <i>General Strategies</i>			c13		
5.2. <i>Function-Oriented (or Structured) Design</i>			c9		
5.3. <i>Data Centered Design</i>			c9		c5.4.1
5.4. <i>Object-Oriented Design</i>			c10		
5.5 <i>User-Centered Design</i>		c9			
5.6. <i>Component-Based Design</i>	c25, c29		c11, c16		c16
5.7. <i>Event-Driven Design</i>					c5.4.2
5.8. <i>Aspect-Oriented Design</i>	c10				c31
5.9. <i>Constraint-Based Design</i>		c11			
5.10. <i>Domain-Driven Design</i>			c13.6.2, c18.3		
5.11. <i>Other Methods</i>					c18-21
6. Software Design Quality Analysis and Evaluation			c7		c24
6.1. <i>Design Reviews and Audits</i>			c5.3		
6.2. <i>Quality Attributes</i>					c24
6.3. <i>Quality Analysis and Evaluation Techniques</i>					c24
6.4. <i>Measures and Metrics</i>			c5, c17		c24.5
6.5. <i>Verification, Validation and Certification</i>					c7-8

FURTHER READINGS

Brooks, *The Design of Design* [3*]

Brooks, one of the pioneers of software engineering, provides a collection of essays and case studies on all aspects of software design.

REFERENCES

- [1*] G. Booch, J. Rumbaugh, and I. Jacobson, *The Unified Modeling Language User Guide*, 2nd edition, Addison-Wesley, 2005.
- [2] J. Bosch, *Design and Use of Software Architectures: Adopting and Evolving a Product-Line Approach*, ACM Press, 2000.
- [3*] F. Brooks, *The Design of Design*, Addison-Wesley, 2010.
- [4*] D. Budgen, *Software Design: Creating Solutions for Ill-Structured Problems*, 3rd Edition CRC Press, 2021.
- [5] E.W. Dijkstra, On the Role of Scientific Thought. 1974. <http://www.cs.utexas.edu/users/EWD/transcriptions/EWD04xx/EWD447.html>.
- [6] M. Galster, D. Weyns, D. Tofan, B. Michalik, and P. Avgeriou, Variability in Software Systems — A Systematic Literature Review, *IEEE Transactions on Software Engineering*, 40(3), 2014.
- [7*] E. Gamma et al., *Design Patterns: Elements of Reusable Object-Oriented Software*, 1st ed, Addison-Wesley, 1994.
- [8] S. Gregor and D. Jones, The Anatomy of a Design Theory, Association for Information Systems, 2007.
- [9] IEEE Std 7000™-2021, IEEE Standard Model Process for Addressing Ethical Concerns during System Design.
- [10] ISO/IEC/IEEE 12207, Systems and Software Engineering — Software Life Cycle Processes.
- [11] ISO/IEC/IEEE 24765:2017 Systems and Software Engineering — Vocabulary, 2nd ed. 2017.
- [12] G. Kiczales et al., Aspect-Oriented Programming, *Proc. 11th European Conf. Object-Oriented Programming (ECOOP 97)*, Springer, 1997.
- [13] T. Kosar, S. Bohra, M. Mernik, Domain-Specific Languages: A Systematic Mapping Study, *Information and Software Technology*, 71, 77-91, 2016.
- [14] D. Luckham, *The Power of Events: an Introduction to Complex Event Processing*, Addison-Wesley, 2002.
- [15] G. Mühl, L. Fiege, and P. Pietzuch, *Distributed Event-Based Systems*, Springer-Verlag, 2006.
- [16] J. Nielsen, *Usability Engineering*, Morgan Kaufman, 1994.
- [17] D.L. Parnas, On the Criteria To Be Used In Decomposing Systems Into Modules, *Communications of the ACM* 15(12), 1053–1058, 1972.
- [18] D.L. Parnas and P.C. Clements, A Rational Design Process: How and Why to fake it, *IEEE Transactions on Software Engineering* 12(2), 251– 257, 1986.
- [19] D.L. Parnas and D.M. Weiss, Active Design Reviews: Principles and Practices, *Journal of Systems & Software* 7, 259–265, 1987
- [20] D.T. Ross, J.B. Goodenough, and A. Irvine, Software Engineering: Process, Principles, and Goals, *IEEE Computer*, May 1975.
- [21*] I. Sommerville, *Software Engineering*, 10th edition, Pearson, 2016.